Java 17 vs Virtual Threads

50 new OS threads per request

The leading explanation for the performance gap

Benchmark findings · 13 November 2026

About nine times the throughput

Screenshot metric	Java 17	Java 21	Run labeled Java 25*
Requests / second	102	**955**	**917**
Average latency	992 ms	105 ms	109 ms
p95 latency	3.54 s	110 ms	115 ms
HTTP failures	0%	0%	0%

9.3× higher throughput for Java 21 in this run

*The right-hand run has conflicting version labels. The screenshot reports totals over a ramp, rather than a constant-load plateau.

The workload spends 100 ms waiting

```
GET /benchmark/concurrent?tasks=50&delay=100
```

1. Each request submits **50 concurrent tasks**.
2. Each task calls `Thread.sleep(100)`.
3. The request waits for **all 50 tasks** to finish.

With enough capacity, response time stays near **100 ms**.

Sleep models waiting. It does not exercise a real database, network dependency, or CPU-heavy workload.

Java 17 creates a pool per request

```
ExecutorService executor =
    Executors.newFixedThreadPool(tasks);
try {
    // Submit 50 tasks and wait for completion
    CompletableFuture.allOf(...).join();
} finally {
    executor.shutdown();
}
```

The next request creates another 50 OS threads.

The pool can reuse workers during its lifetime. This lifecycle prevents reuse across HTTP requests.



Thread creation becomes substantial

≈ 5,100

OS threads created and terminated per second

```
102 requests/s × 50 threads/request
```

Potential demand at 200 active requests

200 × 50 = 10,000 task threads, plus request and JVM threads

These are calculations from the workload and observed throughput. The review did not measure thread-start rates or peak thread counts.

High CPU can come from thread overhead

`Thread.sleep(100)` suspends execution instead of busy-waiting.

- Native thread creation initializes OS and JVM resources.
- Starting and waking workers adds scheduling work.
- Pool shutdown adds thread cleanup on every request.
- Completion coordination and allocation add more overhead.

These costs are likely contributors. A profile must quantify them.

The CPU annotations show 400% vs 40%. On a per-core scale, 400% means about four logical CPUs busy. The source and sampling interval are unknown.

Virtual threads release the carrier while waiting

```
try (ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor()) {
    // Submit the same 50 sleeping tasks
    CompletableFuture.allOf(...).join();
}
```

- The JVM schedules virtual threads on shared OS carriers.
- During sleep, a virtual thread can unmount from its carrier.
- Many waiting tasks can share a much smaller OS thread set.

Observed average latency stays near the intended 100 ms wait.



Slower responses reduce k6 throughput

Each virtual user sends a request, waits for its response,
then begins the next iteration.

```
Throughput ≈ active users / response time
```

- Longer responses let each user complete fewer requests.
- The screenshot ramps toward **200 users over one minute**.
- Its average mixes lighter load with the busiest part of the run.

The formula describes a stable closed workload with negligible think time. It is not an exact calculation for this changing ramp.

The comparison changes more than the JDK

Dimension	Java 17 app	Java 21/25 apps
Task executor	Fresh platform pool	Virtual thread per task
Spring virtual threads	Disabled by default	Explicitly enabled
Blocking work	50 sleeps × 100 ms	50 sleeps × 100 ms

- The threading design is a major difference in this test.
- The review has no CPU profile or verified runtime version log.
- A single ramp does not establish sustained production capacity.

A controlled comparison can isolate the costs

Comparison	What it isolates
Fresh vs shared platform pool on JDK 17	Thread lifecycle overhead
Same platform design on JDK 17, 21, 25	JDK-version effects
Platform vs virtual tasks on one newer JDK	Task-thread strategy
Same task executor, Spring virtual on vs off	Request-thread strategy

Warm-up, then a fixed-load plateau with matching resources.

Collect CPU profiles, thread-start rates, and scheduling evidence. A bounded shared pool still queues work when every worker is sleeping.

The findings support a clear next step

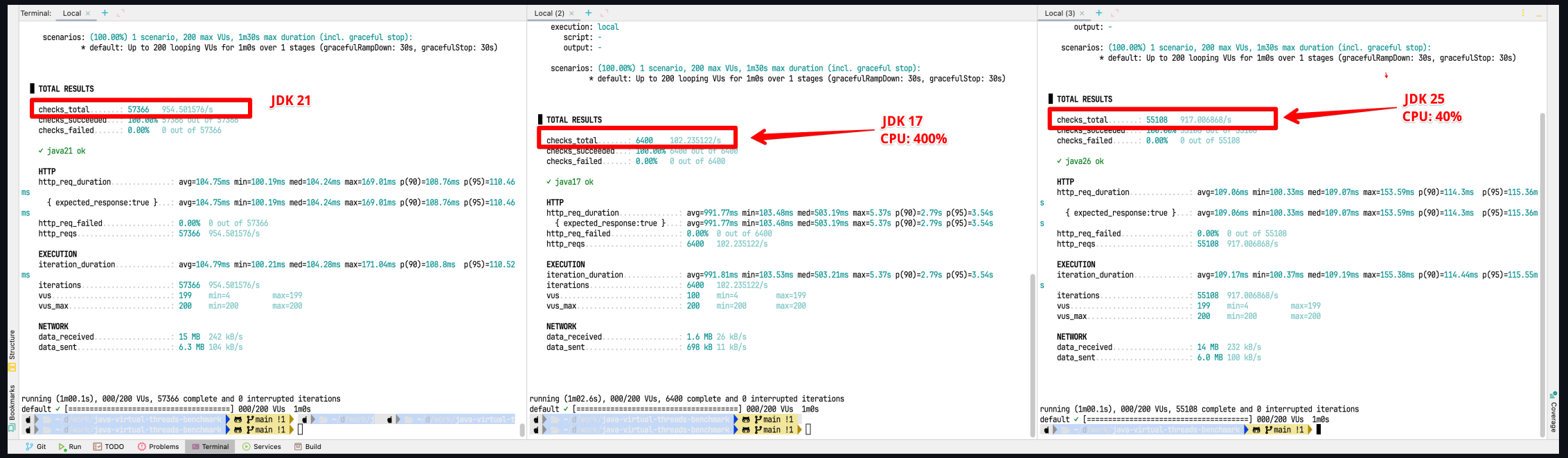
1. **Reuse platform workers** to measure how much thread creation costs.
2. **Keep virtual threads** as the comparison for many waiting tasks.
3. **Profile under sustained load** before assigning the 400% CPU cost.

The main finding is the cost of **50 fresh OS threads per request**.

The evidence does not establish a general 9× JDK speedup or a comparable benefit for CPU-heavy work.



Appendix: original benchmark evidence



Original screenshot, preserved in full. The results slide extracts its metrics at presentation size.

Version labels conflict in the right-hand run. CPU values are user annotations. Both limitations remain open until runtime verification and profiling.